



GuardianAI

Autonomous AI Agent Safety & Permission Control System

AMD AI DEVMASTER HACKATHON 2026

Team Member

Poovalingam M

Tech Stack

React.js · Spring Boot · Java · Python · PostgreSQL · Docker · AMD ROCm

Document

Stage 1 — Presentation Deck · Version 1.0

Table of Contents

01	Problem Statement	3
02	Proposed Solution	4
03	System Architecture	5
04	Workflow	7
05	Technology Stack	8
06	Security Features	9
07	Future Enhancements & Conclusion	10

Problem Statement

Background

Modern AI agents can autonomously access APIs, databases, cloud resources, and enterprise systems. While this increases productivity, it also introduces security risks such as unauthorized actions, prompt injection, excessive permissions, and data leakage.

Challenges

- AI agents may execute dangerous commands.
- Lack of permission validation before execution.
- No centralized policy enforcement.
- Limited transparency into AI decisions.
- Inadequate audit logging.

Proposed Solution

GuardianAI Overview

GuardianAI introduces a security layer between users and AI agents — evaluating, governing, and logging every action before it ever reaches a live system.

Objectives

- Analyze user requests.
- Detect malicious or risky instructions.
- Enforce permission-based policies.
- Execute only approved actions.
- Record every action in audit logs.

System Architecture

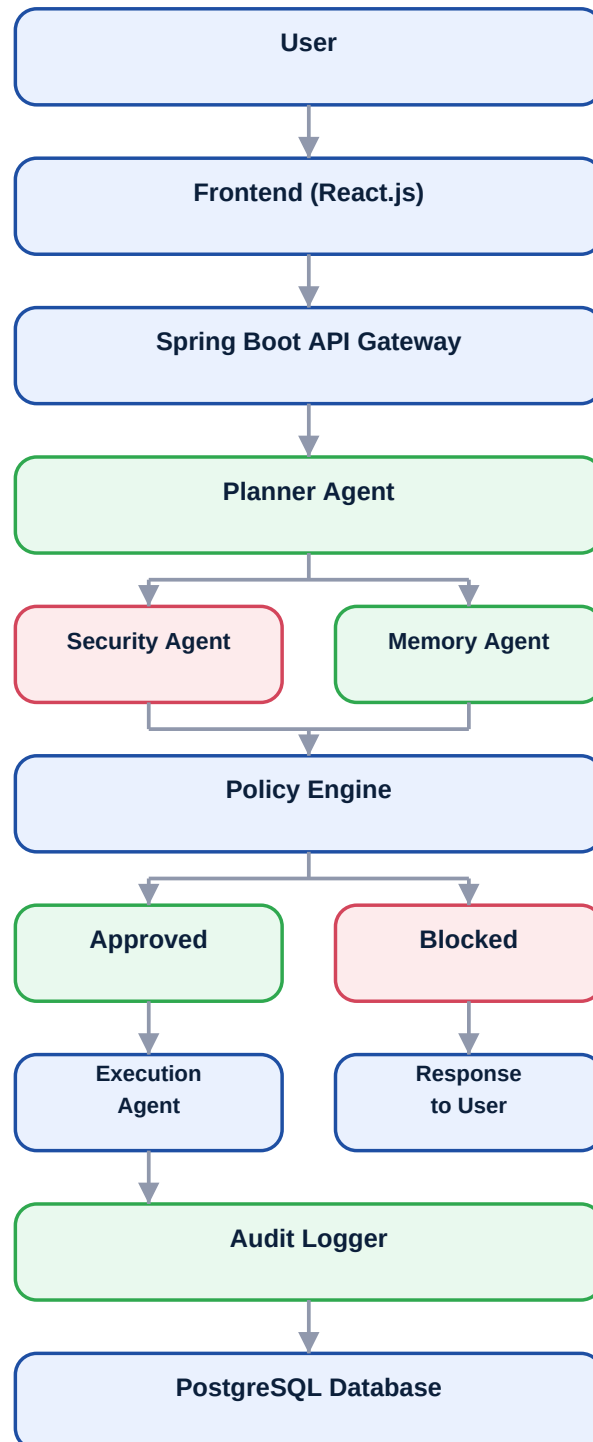


Figure — GuardianAI request-evaluation pipeline, from user request to audit log.

Architecture — Component Overview

Frontend (React)	User-facing interface for submitting requests and reviewing agent activity and audit trails.
Spring Boot API	Central gateway that authenticates requests and routes them into the agent pipeline.
Planner Agent	Interprets the user's intent and breaks it into an actionable task plan.
Security Agent	Screens the plan for risky operations, prompt injection, and unsafe patterns.
Memory Agent	Supplies relevant context and history so decisions are consistent and well-informed.
Policy Engine	Validates the plan against configurable permission rules and assigns a risk level.
Approved / Blocked	Binary gate — approved actions proceed, blocked ones are halted and reported.
Execution Agent	Carries out approved actions against the target system or tool.
Audit Logger	Immutably records every request, decision, and outcome.
Database (PostgreSQL)	Persists policies, audit logs, and agent state.

Workflow

- | | |
|---|---|
| 1 | Step 1 — User submits a request. |
| 2 | Step 2 — Planner Agent identifies the task. |
| 3 | Step 3 — Security Agent evaluates the risk. |
| 4 | Step 4 — Memory Agent retrieves relevant context. |
| 5 | Step 5 — Policy Engine validates permissions. |
| 6 | Step 6 — If approved, Execution Agent performs the action. |
| 7 | Step 7 — Audit Logger records the request, decision, and outcome. |

Technology Stack

Layer	Technology	Why It Was Selected
Frontend	React.js, TypeScript, Tailwind CSS	Fast, component-driven UI development with strong type safety and consistent styling for the audit dashboard.
Backend	Spring Boot, Java	Enterprise-grade reliability, a mature security ecosystem (Spring Security), and strong support for robust REST APIs.
AI Layer	LangChain, Local LLM	Enables multi-agent orchestration (Planner, Security, Memory) while keeping all inference local — no paid third-party API dependency.
Database	PostgreSQL	ACID-compliant relational storage well suited for immutable audit logs and structured policy definitions.
Authentication	JWT	Stateless, scalable token-based authentication that integrates cleanly with the Spring Boot API layer.
Containerization	Docker	Guarantees reproducible, one-command deployment across judging and demo environments.
GPU Acceleration	AMD Radeon GPU, ROCm	Accelerates local LLM inference and risk-classification workloads without relying on proprietary CUDA-only tooling.

Security Features

Role-Based Access Control (RBAC)	Restricts agent capabilities based on user and agent roles, minimizing excessive permissions.
Prompt Injection Detection	Screens incoming instructions for manipulation attempts before they reach the Planner Agent.
Risk Scoring	Assigns a quantitative risk level to every requested action for consistent decision-making.
Policy Validation	Checks every action against configurable, centrally managed permission policies.
Human Approval for High-Risk Actions	Routes sensitive operations to a human reviewer instead of auto-executing them.
Explainable AI Decisions	Every allow/block decision is accompanied by a clear, human-readable reason.
Immutable Audit Logs	Every request, decision, and outcome is permanently recorded for accountability.

Together, these capabilities ensure that AI agent behavior remains predictable, governed, and fully accountable — the foundation of trustworthy autonomous AI.

Future Enhancements & Conclusion

Future Work

- Adaptive AI policy learning.
- Zero Trust AI architecture.
- Multi-agent collaboration.
- Enterprise IAM integration.
- Multi-cloud deployment.
- Advanced GPU optimization with ROCm.

Conclusion

GuardianAI provides a secure, transparent, and scalable framework for governing autonomous AI agents. By combining multi-agent reasoning, policy enforcement, and comprehensive auditing, it helps organizations adopt AI safely while maintaining accountability.